

Segurança em Computação

Vulnerabilidades Modernas

Prof. Jean Everson Martina

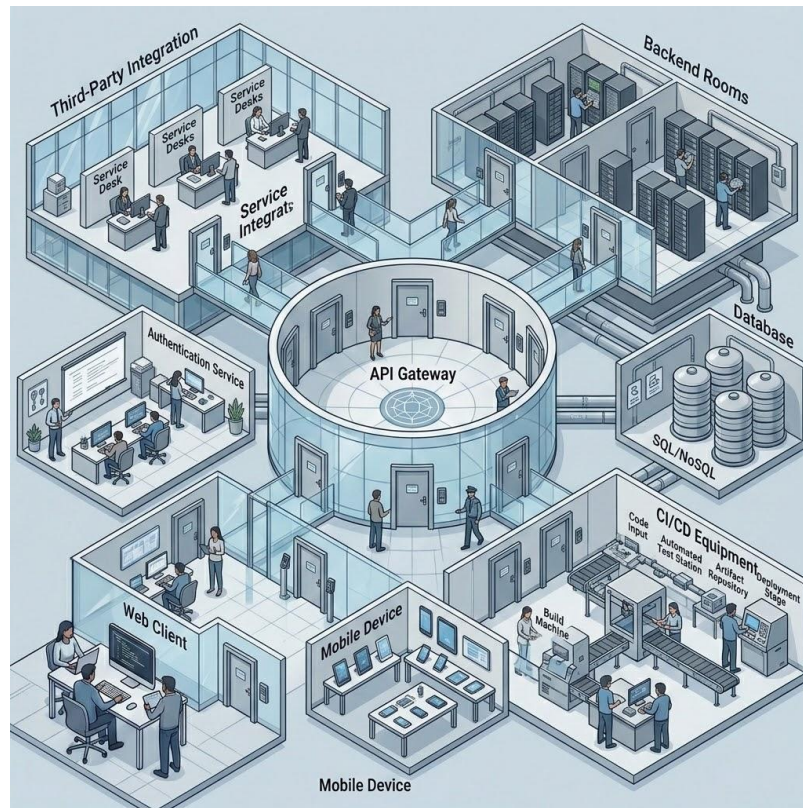
Aplicações modernas falham de formas modernas



- Aplicações modernas combinam frontend web, apps móveis, APIs, backend, cloud e automação.
- As falhas de segurança raramente estão em um único ponto isolado do sistema.
- Vulnerabilidades surgem na interação entre autenticação, autorização, integrações e infraestrutura.
- O modelo atual exige enxergar segurança como problema de arquitetura e engenharia.
- O foco da aula é entender como aplicações modernas quebram e por que isso continua acontecendo.

A superfície de ataque explodiu

- Cada aplicação moderna expõe múltiplos pontos de entrada, como navegadores, apps móveis, APIs públicas e integrações externas.
- O backend normalmente acessa banco de dados, sistemas de autenticação, filas, cache e serviços de terceiros.
- Pipelines, repositórios, tokens e dependências também fazem parte da superfície de ataque.
- Quanto maior o número de componentes e integrações, maior a chance de erro de configuração ou confiança excessiva.
- Segurança moderna exige mapear onde a aplicação conversa, com quem ela confia e o que cada parte pode fazer.



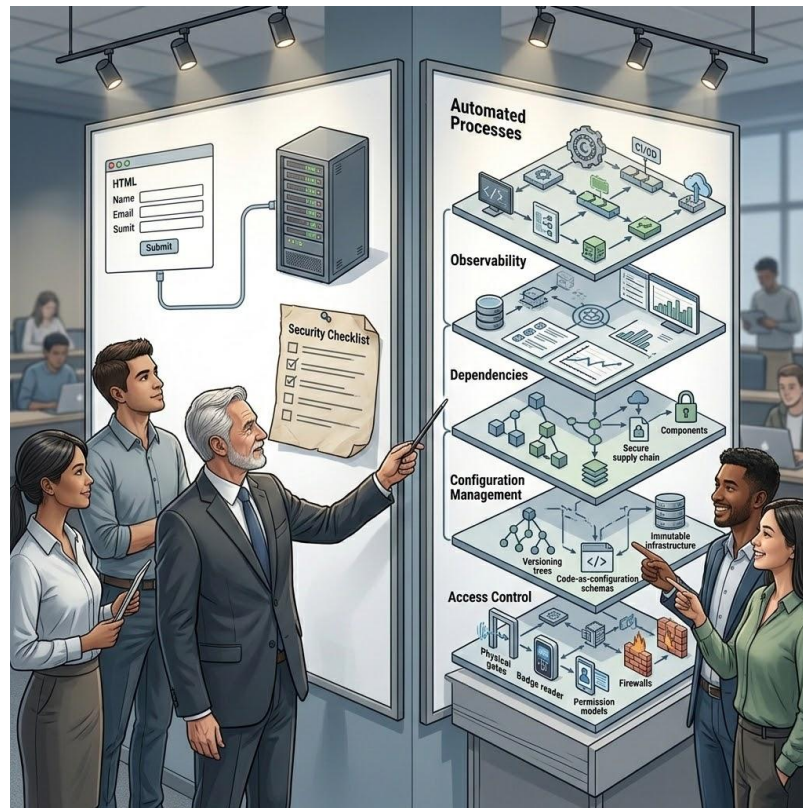
OWASP moderno: um mapa de falhas de engenharia



- OWASP não deve ser lido como uma coleção de payloads ou ataques isolados.
- O valor do OWASP está em organizar classes recorrentes de falhas em aplicações modernas.
- A referência hoje envolve diferentes famílias, como OWASP Top 10, OWASP API Security Top 10 e OWASP Mobile Top 10.
- O objetivo é compreender padrões de erro de engenharia, não decorar nomes de ataques.
- A maior parte das vulnerabilidades modernas reflete decisões ruins de design, integração, validação e operação.

O que mudou no OWASP atual

- O OWASP moderno enfatiza menos ataques clássicos isolados e mais falhas sistêmicas de engenharia.
- Broken Access Control continua sendo uma das falhas mais críticas e frequentes.
- Security Misconfiguration e Software Supply Chain Failures ganharam peso no cenário atual.
- Injection continua relevante, mas agora convive com APIs, templates, SSRF, supply chain e IA.
- O mapa atual mostra que o problema moderno está em autorização, configuração, dependências, observabilidade e processo.



Broken Access Control: o erro que nunca desaparece



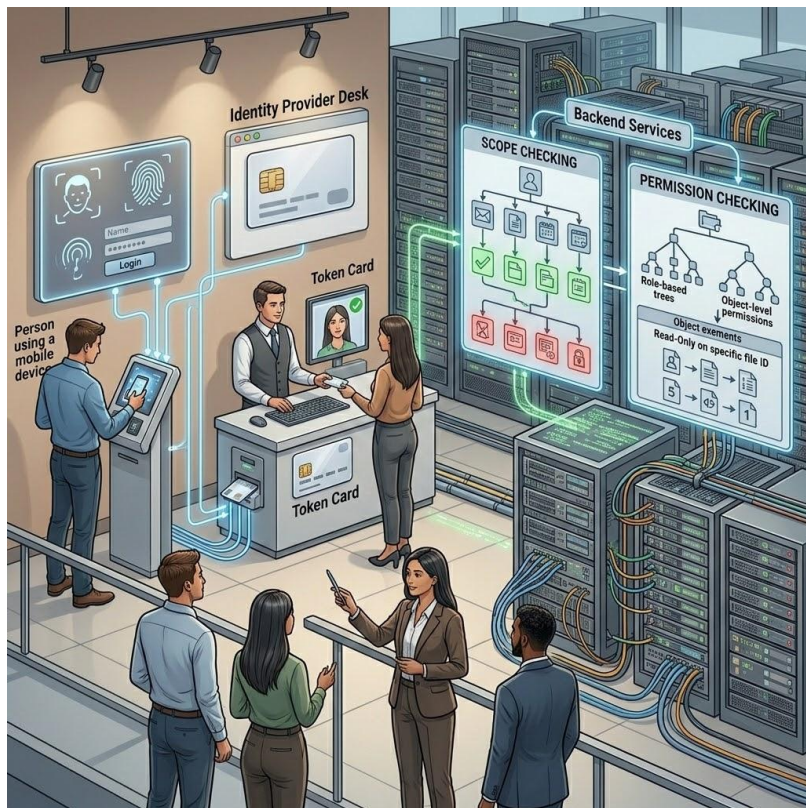
- Broken Access Control acontece quando um usuário autenticado consegue acessar recursos ou executar ações além do que deveria.
- O erro normalmente não está na ausência de login, mas na ausência de verificação correta de permissão.
- Isso pode gerar leitura indevida, alteração não autorizada, exclusão de dados ou escalada de privilégios.
- A falha é comum porque muitos sistemas validam identidade, mas não validam se aquele usuário pode fazer aquela ação naquele objeto.
- Em sistemas distribuídos, autorização precisa ser aplicada no backend, de forma consistente e verificável.

APIs, BOLA e IDOR

- Em APIs, uma das falhas mais importantes é BOLA, ou Broken Object Level Authorization.
- Ela ocorre quando a aplicação acessa objetos por identificadores e não valida corretamente se aquele usuário pode acessar aquele objeto.
- Isso aparece frequentemente em URLs e endpoints como `/users/123/orders` ou `/documents/456`.
- Trocar o identificador e obter acesso a outro recurso é um caso típico de IDOR, ou Insecure Direct Object Reference.
- UUIDs e identificadores longos não resolvem o problema sozinhos. O que resolve é autorização correta no backend.



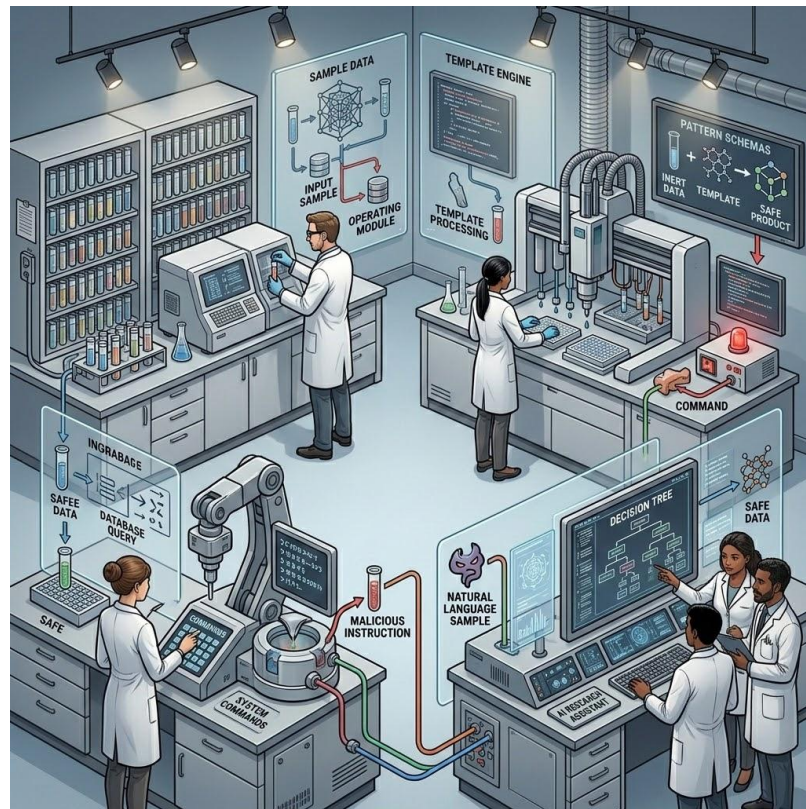
Autenticação moderna também falha



- Aplicações modernas usam com frequência OAuth2, OpenID Connect, JWT, MFA e sessões distribuídas.
- Esses mecanismos melhoram a arquitetura, mas não garantem segurança automaticamente.
- Tokens longos demais, refresh tokens expostos e armazenamento inseguro em browser ou app móvel aumentam o risco.
- Muitas falhas surgem quando o sistema autentica corretamente, mas trata o token como se ele resolvesse sozinho todos os problemas de autorização.
- Autenticação forte reduz risco, mas segurança real depende também de sessão segura, escopo correto e verificação consistente.

Injection ainda existe, mas mudou de forma

- A família de Injection continua relevante porque muitos sistemas ainda confundem dados com instruções.
- O caso clássico é SQL Injection, mas hoje isso se expande para NoSQL Injection, Command Injection e Template Injection.
- Em sistemas modernos também aparecem abusos em filtros, serialização, expressões e mecanismos de template.
- Em ferramentas assistidas por IA, o mesmo raciocínio reaparece em Prompt Injection, quando entradas manipulam o comportamento do modelo.
- A raiz do problema continua a mesma: a aplicação aceita entrada não confiável e a interpreta com poder excessivo.



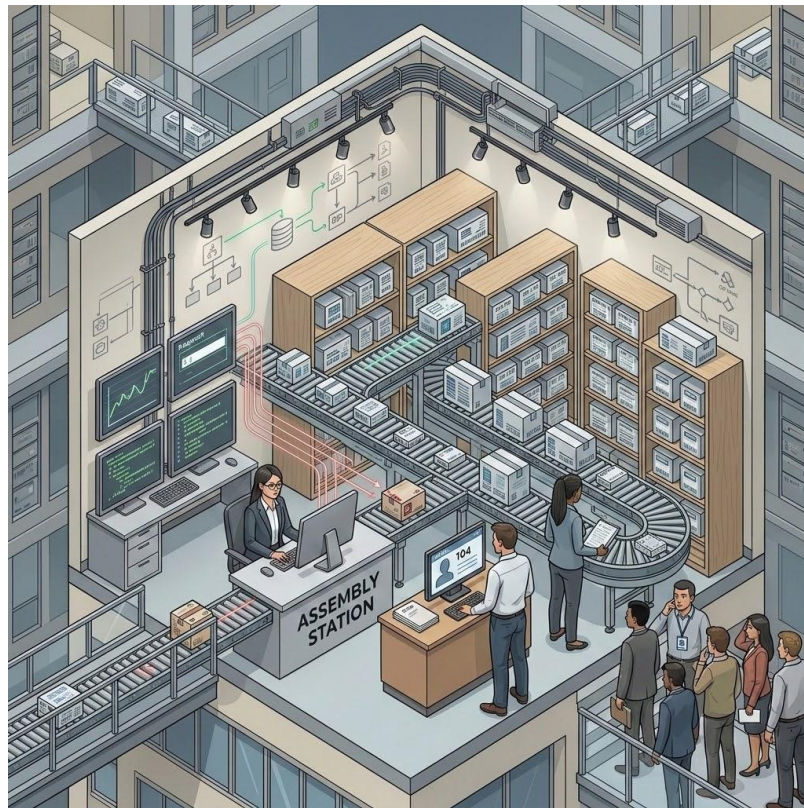
SSRF e confiança excessiva no servidor



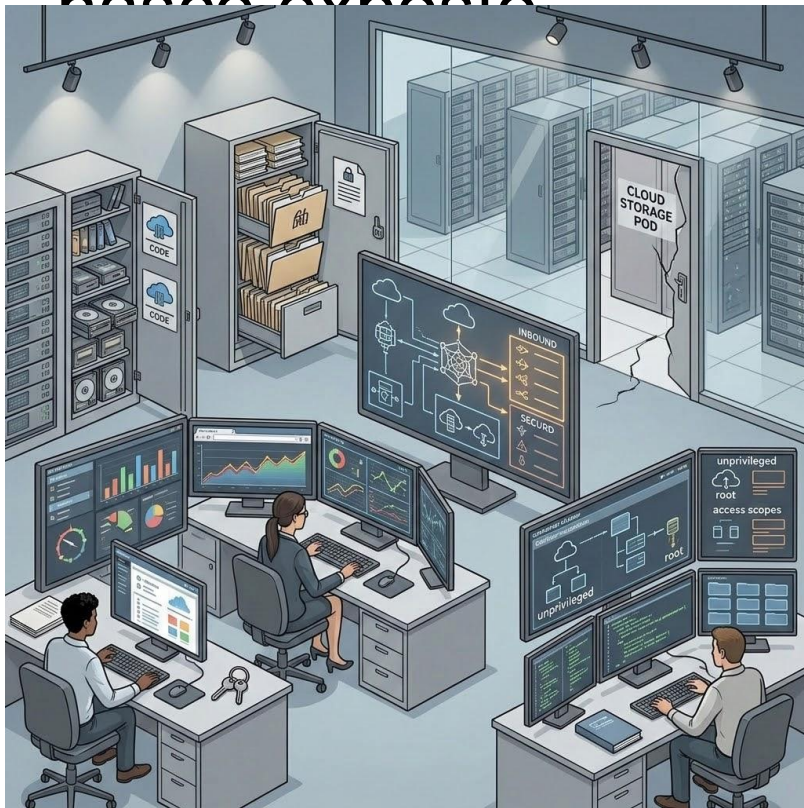
- SSRF, ou Server Side Request Forgery, ocorre quando o servidor é induzido a fazer requisições em nome do atacante.
- Isso é especialmente perigoso porque o servidor pode alcançar serviços internos que o atacante não conseguiria acessar diretamente.
- Em ambientes cloud, SSRF pode alcançar endpoints de metadata e expor credenciais ou informações sensíveis.
- A falha costuma nascer de funcionalidades que aceitam URLs ou recursos externos sem validação adequada.
- Mitigação envolve validação rigorosa, allowlists, controle de egress e isolamento entre serviços.

Supply chain e dependências vulneráveis

- Grande parte do software moderno é composta por bibliotecas, frameworks e pacotes externos.
- Isso cria dependências diretas e transitivas que muitas vezes não são plenamente conhecidas pelo time de desenvolvimento.
- Casos como Log4Shell mostraram que uma única biblioteca pode afetar milhares de sistemas ao mesmo tempo.
- Ecossistemas como npm e PyPI também sofrem com typosquatting, takeover de pacotes, dependências maliciosas e manutenção abandonada.
- Segurança moderna precisa perguntar não só o que desenvolvemos, mas também o que estamos executando sem perceber.



Security Misconfiguration: quando o sistema já



- Muitas invasões não exploram bugs sofisticados, mas serviços, recursos e permissões mal configurados.
- Exemplos comuns incluem buckets públicos, CORS permissivo, painéis administrativos expostos, debug em produção e containers privilegiados.
- Em cloud, erros de security groups, políticas excessivamente abertas e variáveis com secrets também ampliam o risco.
- Em APIs, configuração insegura pode significar excesso de exposição, ausência de rate limiting ou headers inadequados.
- Security Misconfiguration é um problema de engenharia operacional e de ausência de controles automáticos.

Logging, alerting e observabilidade insuficiente

- Segurança moderna depende de logs úteis, alertas confiáveis e visibilidade operacional.
- Sem registro adequado de ações, origem, identidade e resultado, incidentes se tornam difíceis de investigar.
- Logging insuficiente transforma falhas exploradas em problemas invisíveis ou descobertos tarde demais.
- Alertas ruins também prejudicam a defesa, porque excesso de ruído descredibiliza o processo de monitoramento.
- Observabilidade é a ponte entre vulnerabilidade, detecção, resposta e melhoria contínua.



Por que essas falhas continuam acontecendo



- Pressão por entrega rápida frequentemente supera a preocupação com requisitos de segurança.
- Equipes confiam demais em frameworks, bibliotecas e plataformas, como se eles eliminassem a necessidade de validação.
- Adoção acrítica de IA e automação pode ampliar a produção de código e configuração inseguros.
- Segurança ainda é tratada por muitas organizações como etapa final, não como parte do processo de desenvolvimento.
- As vulnerabilidades persistem porque o processo que as gera continua ativo, fragmentado e mal governado.

Fechamento: da vulnerabilidade ao pipeline seguro

- Nesta aula vimos que aplicações modernas falham principalmente por erros de autorização, validação, configuração, dependências e observabilidade.
- Muitas dessas falhas surgem antes da produção e poderiam ser identificadas com práticas melhores de engenharia.
- Outras exigem monitoramento, testes dinâmicos e validação contínua em runtime.
- A próxima aula tratará justamente de como o pipeline pode incorporar segurança por meio de SAST, DAST, IAST, SCA e secrets scanning.
- A pergunta agora não é apenas como a aplicação falha, mas como impedir que essas falhas cheguem à produção.

